

Camunda 8.9 PoC on an existing ECS cluster and ALB

No EFS, no Aurora · private subnets · access from the corporate laptop

What this gives you

One Fargate task runs Camunda's Orchestration Cluster (Zeebe, Operate, Tasklist, Admin, REST API) with an embedded H2 database, plus Connectors as a second container. It joins your existing ECS cluster as a new service and sits behind your existing ALB, so you reach it at a stable ALB address instead of a task IP.

Limitations (PoC only):

- All data lives on the task's own temporary disk. Every restart or redeploy starts from empty: deployed processes, instances and users are gone.
- One broker, no high availability. H2 is supported by Camunda for development, testing and evaluation only.
- Zeebe gRPC (port 26500) cannot go through an ALB on a plain HTTP listener. Use the REST API through the ALB instead (section 8).

Files used:

File	Purpose
camunda-poc-h2-taskdef.json	Task definition: orchestration + connectors containers
camunda-poc-h2-service.json	ECS service on your existing cluster, registered with the ALB target group

1. Choose how the ALB routes to Camunda

Camunda's web apps expect to be served from the root path `/`, so give the PoC its own listener port or its own host name rather than a sub-path of another app.

Option	What you add to the ALB	Needs	Recommendation
A. Dedicated listener port	New listener, e.g. HTTP 8088, default action → Camunda target group	Nothing else	Use this for the PoC. No DNS, no change to existing rules
B. Host-based rule	Rule on the existing listener: Host header <code>camunda-poc.<corp-domain></code> → Camunda target group	A DNS record pointing to the ALB, and a certificate covering that name if the listener is HTTPS	Use if the security team won't open a new port
C. Path-based rule (<code>/camunda/*</code>)	Rule on the existing listener by path	Context-path changes inside Camunda	Avoid: the UI and APIs break without extra configuration

Confirm the ALB is reachable from the corporate laptop. An **internal** ALB in the private subnets is ideal; an internet-facing one must already allow your corporate IP range.

2. Create the log group

CloudWatch → Log groups → **Create log group** → name `/ecs/camunda-poc-h2` → retention 7 days.

3. Security groups

New task security group `camunda-poc-tasks` in the cluster's VPC:

Direction	Port	Source / destination	Why
Inbound	TCP 8080	The ALB's security group	Web UI and REST API traffic
Inbound	TCP 9600	The ALB's security group	Target group health checks
Inbound (optional)	TCP 26500	Corporate CIDR	Direct gRPC to the task IP (section 8)
Outbound	All	0.0.0.0/0 (default)	ECR, CloudWatch Logs via your VPC endpoints

Existing ALB security group (change carefully, it is shared):

- Option A: add inbound **TCP 8088** from the corporate CIDR.
- Option B: nothing to add if the listener port is already open to the corporate CIDR.

- If the ALB group's outbound rules are restricted, add outbound **TCP 8080 and 9600** to `camunda-poc-tasks`.

4. Create the target group

EC2 → Target groups → **Create target group**:

Setting	Value
Target type	IP addresses
Name	<code>camunda-poc-h2-tg</code>
Protocol / port	HTTP / 8080
VPC	The cluster's VPC
Protocol version	HTTP1
Health check path	<code>/actuator/health/readiness</code>
Advanced → Health check port	Override → 9600
Healthy / unhealthy threshold	2 / 2
Timeout / interval	5 s / 30 s
Success codes	200

Skip registering targets (ECS does it) → **Create**. Then open the target group → **Attributes** → **Edit**:

- **Stickiness**: on, load balancer generated cookie, 1 day (keeps the web UI session on the same task once you scale).
- **Deregistration delay**: 30 s.

5. Attach the target group to the ALB

Option A: dedicated port

EC2 → Load balancers → your ALB → **Listeners and rules** → **Add listener**:

- Protocol **HTTP**, port **8088** (or any free port the corporate network allows).
- Default action: **Forward to** `camunda-poc-h2-tg`.
- If the ALB already uses HTTPS with a corporate certificate, you can make this listener HTTPS on the same port with that certificate instead.

Option B: host-based rule

Your ALB → the existing listener (usually HTTPS 443) → **Manage rules** → **Add rule**:

- Condition: **Host header** = `camunda-poc.<corp-domain>`.
- Action: **Forward to** `camunda-poc-h2-tg`.
- Priority: any unused number lower than any catch-all rule.
- Ask the DNS team for a **CNAME** `camunda-poc.<corp-domain>` → the ALB's DNS name. The listener's certificate must cover that name.

6. Register the task definition

1. In `camunda-poc-h2-taskdef.json` replace `<ACCOUNT_ID>`, `<REGION>`, `<EXISTING_ECS_TASK_EXECUTION_ROLE>` and both `<CHOOSE_A_PASSWORD>` entries (same value).
2. ECS → Task definitions → **Create new task definition** → **Create new task definition with JSON** → paste → **Create**.

7. Create the service

Console: ECS → Clusters → your cluster → Services → **Create**:

Section	Setting
Compute	Launch type FARGATE , platform LATEST
Deployment configuration	Service · family <code>camunda-poc-h2</code> · desired tasks 1
Deployment options	Min running 0 % , max running 100 % ; circuit breaker with rollback on
Networking	Cluster VPC · private subnets · security group <code>camunda-poc-tasks</code> · public IP off
Load balancing	Application Load Balancer → Use an existing load balancer → your ALB · container orchestration <code>8080:8080</code> · Use an existing listener (8088 for option A, 443 for option B) · Use an existing target group → <code>camunda-poc-h2-tg</code>
Health check grace period	300 seconds

CLI: fill the placeholders in `camunda-poc-h2-service.json` (cluster, subnets, security group, target group ARN), then:

```
aws ecs create-service --cli-input-json file://camunda-poc-h2-service.json
```

The target group must already be attached to a listener (step 5), or the service creation is rejected.

8. Verify and use it

1. ECS → the service → **Tasks**: one task, status **Running**, health **Healthy** after about 2 to 3 minutes.
2. EC2 → Target groups → `camunda-poc-h2-tg` → **Targets**: one target **healthy**.
3. Open the web UI from the corporate laptop and log in as `admin`:
 - Option A: `http://<alb-dns-name>:8088/`
 - Option B: `https://camunda-poc.<corp-domain>/`
4. Check the REST API:

```
curl -u admin:<password> http://<alb-dns-name>:8088/v2/topology
```

5. Deploy a BPMN file through the REST API (works through the ALB, no gRPC needed):

```
curl -u admin:<password> \
  -F "resources=@my-process.bpmn" \
  http://<alb-dns-name>:8088/v2/deployments
```

gRPC (job workers, zbctl, older Modeler setups): an ALB only carries gRPC on an HTTPS listener with a gRPC target group. For the PoC, connect directly to the task instead: allow TCP 26500 from the corporate CIDR on `camunda-poc-tasks` (section 3) and use `http://<task-private-ip>:26500`. The task IP is under the task → **Configuration** → **Private IP**, and changes on every restart.

9. Troubleshooting

Symptom	Likely cause	Fix
Target stays unhealthy , task keeps restarting	ALB can't reach port 9600, or the health-check port wasn't overridden	Allow 9600 from the ALB security group; set the target group health-check port to 9600
Task stops before becoming healthy	Grace period too short for the first start	Keep the health check grace period at 300 s
<code>CannotPullContainerError</code>	ECR URI or VPC endpoint issue	Check the image URI and that the ECR and S3 endpoints serve these subnets
Browser times out on the ALB URL	Listener port not open to the corporate CIDR, or the ALB isn't reachable from the laptop	Check the ALB security group and that the ALB is internal or allows your range
Login works, then you're logged out at random	Stickiness off with more than one task	Turn stickiness on, or keep desired count at 1
Connectors container unhealthy	Orchestration not healthy yet, or password mismatch	Check its log stream in <code>/ecs/camunda-poc-h2</code> ; both passwords must match
Everything you deployed is gone	Task restarted (expected with this design)	Redeploy; for persistence you need EFS + Aurora (or another database)

10. Clean up

1. ECS → the service → **Delete service** (force delete if it still has a running task).

2. ALB → delete the **8088 listener** (option A) or the **host-header rule** (option B). Leave the rest of the ALB alone.
3. EC2 → delete the target group `camunda-poc-h2-tg`, then the security group `camunda-poc-tasks`, and remove any rules you added to the ALB's security group.
4. ECS → Task definitions → `camunda-poc-h2` → deregister the revisions.
5. CloudWatch → delete the log group `/ecs/camunda-poc-h2`.